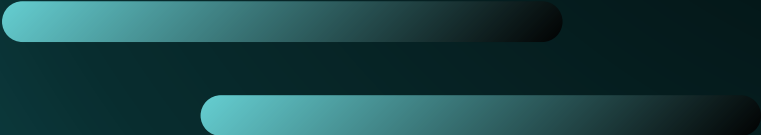


DATA STRUCTURES & ALGORITHMS

COMMUNITY DETECTION



SAANVI TS - 24B4501

SHIVAM SONI - 24B4525

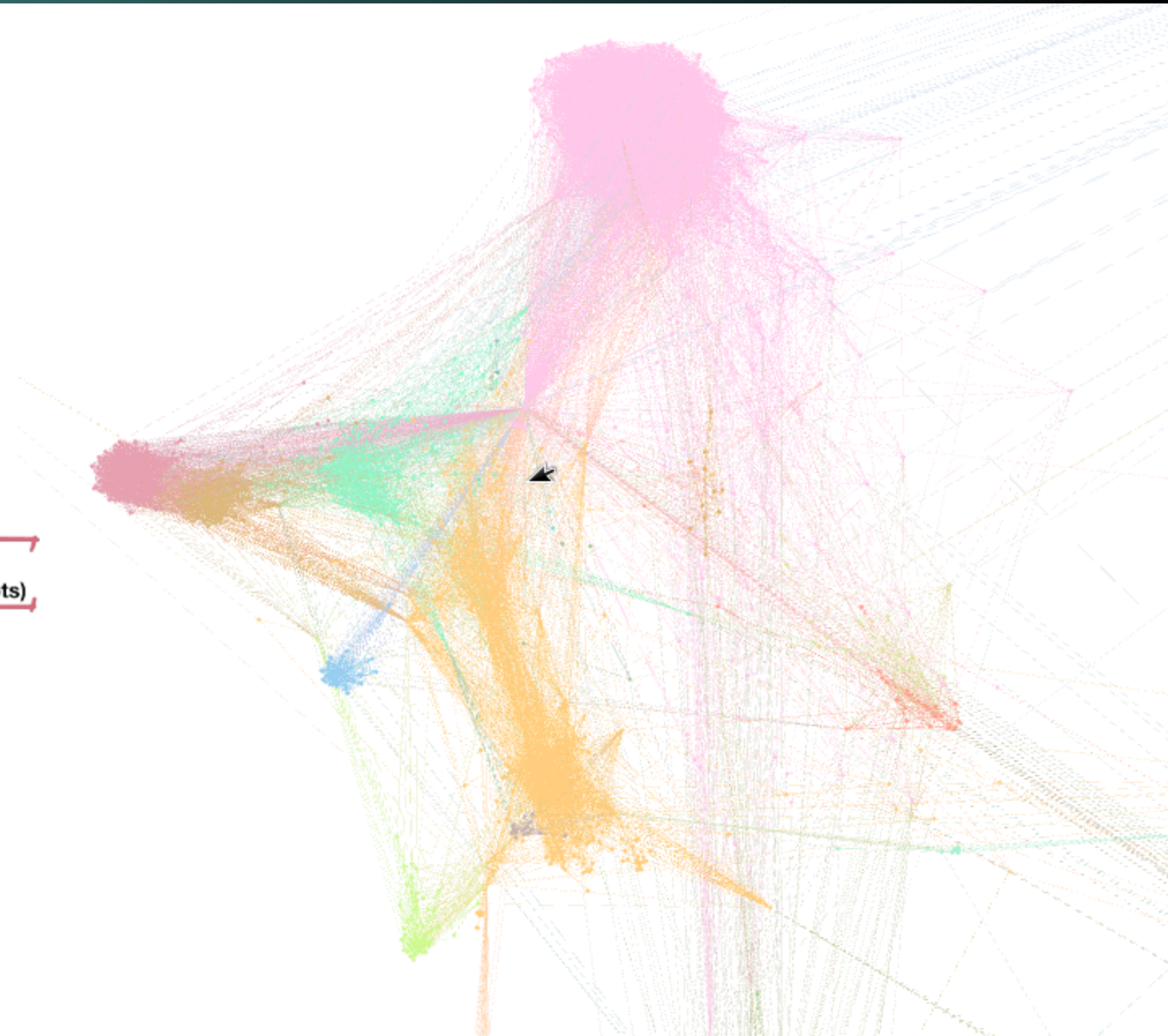
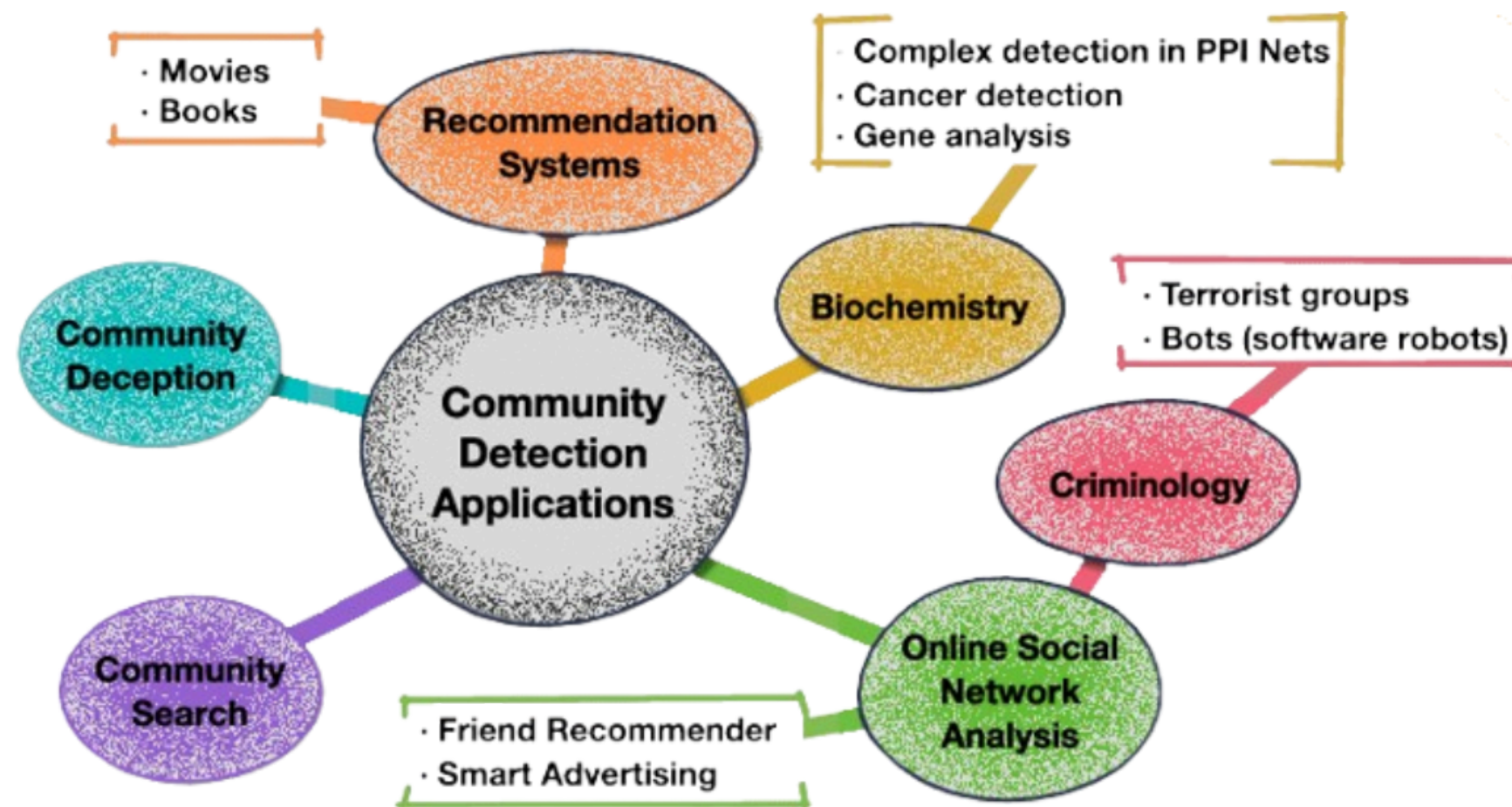
ANANT ASATI - 24B4530

WHAT, HOW and WHY - Community Detection

What is a Community?

A community is a subset of nodes where the internal connections are stronger or more frequent than the external connections to other nodes in the network.

Why Community Detection?



WHAT, HOW and WHY - Community Detection

How do we detect Communities?

- search algos?
- random walkers?
- greedy approach?

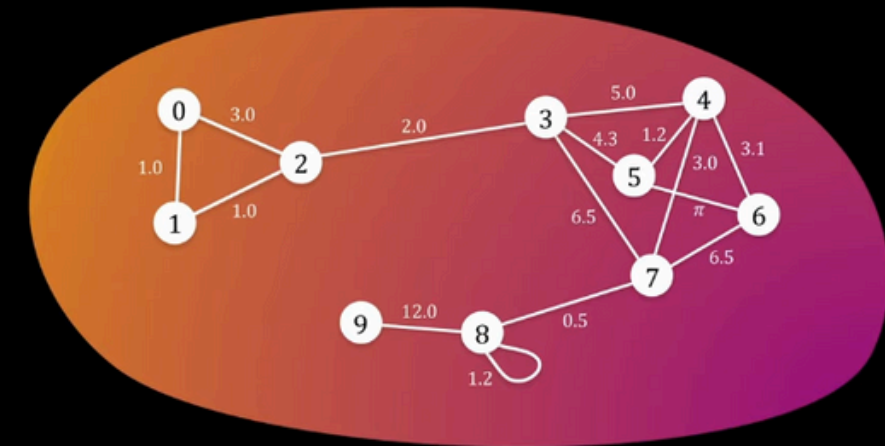
How do we know our Community Distribution is good?

- enter Modularity - the strength of division

$$Q(\mathcal{C}) := \frac{1}{2m} \sum_{c \in \mathcal{C}} \sum_{u \in c} \sum_{v \in c} \left(\underbrace{A_{uv}}_{\text{Actual}} - \underbrace{\frac{k_u k_v}{2m}}_{\text{Random baseline}} \right)$$

A high Q (closer to 1) means the internal connections are much denser than expected by chance.

Modularity



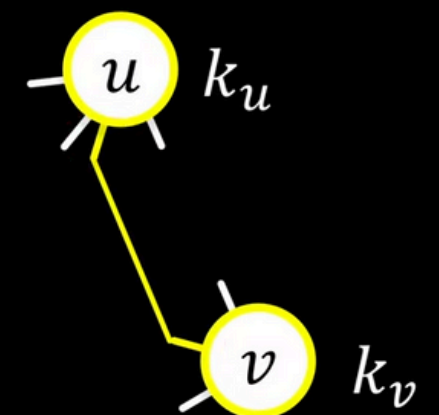
Fraction of edges that fall within the given communities

Actual

— Fraction of edges that fall within the given communities if edges were distributed at random

Probability that these two edge stubs connect?

$$\frac{1}{2m - 1}$$



can we formulate a greedy algorithm using this?

The Louvain Algorithm

Goal - Modularity Maximization (Q)

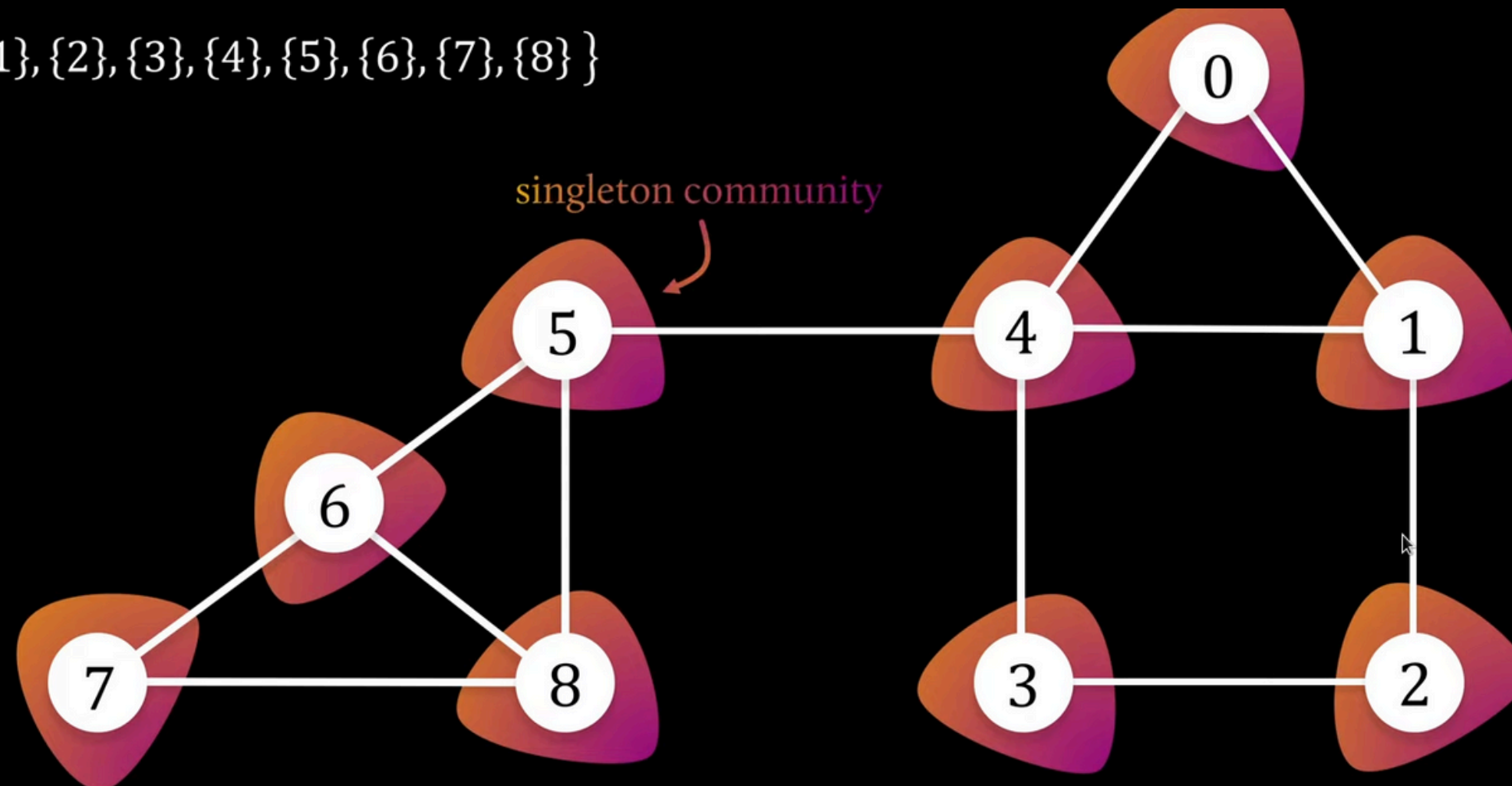
- Louvain is a **greedy, hierarchical, agglomerative** algorithm for community detection.

The Two-Phase Iterative Process

Phase 1: Local Modularity Optimization (Greedy)

- The process begins by treating every single node as its own community.
- For each node, the algorithm calculates the **modularity gain** (Q) that would result from moving it to a neighboring community.
- The node is then moved to the community that provides the largest positive gain.
- This phase is repeated until no single node move can further increase the overall modularity score.

$$\mathcal{C} = \{\{0\}, \{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}, \{8\}\}$$



The Louvain Algorithm

The Two-Phase Iterative Process

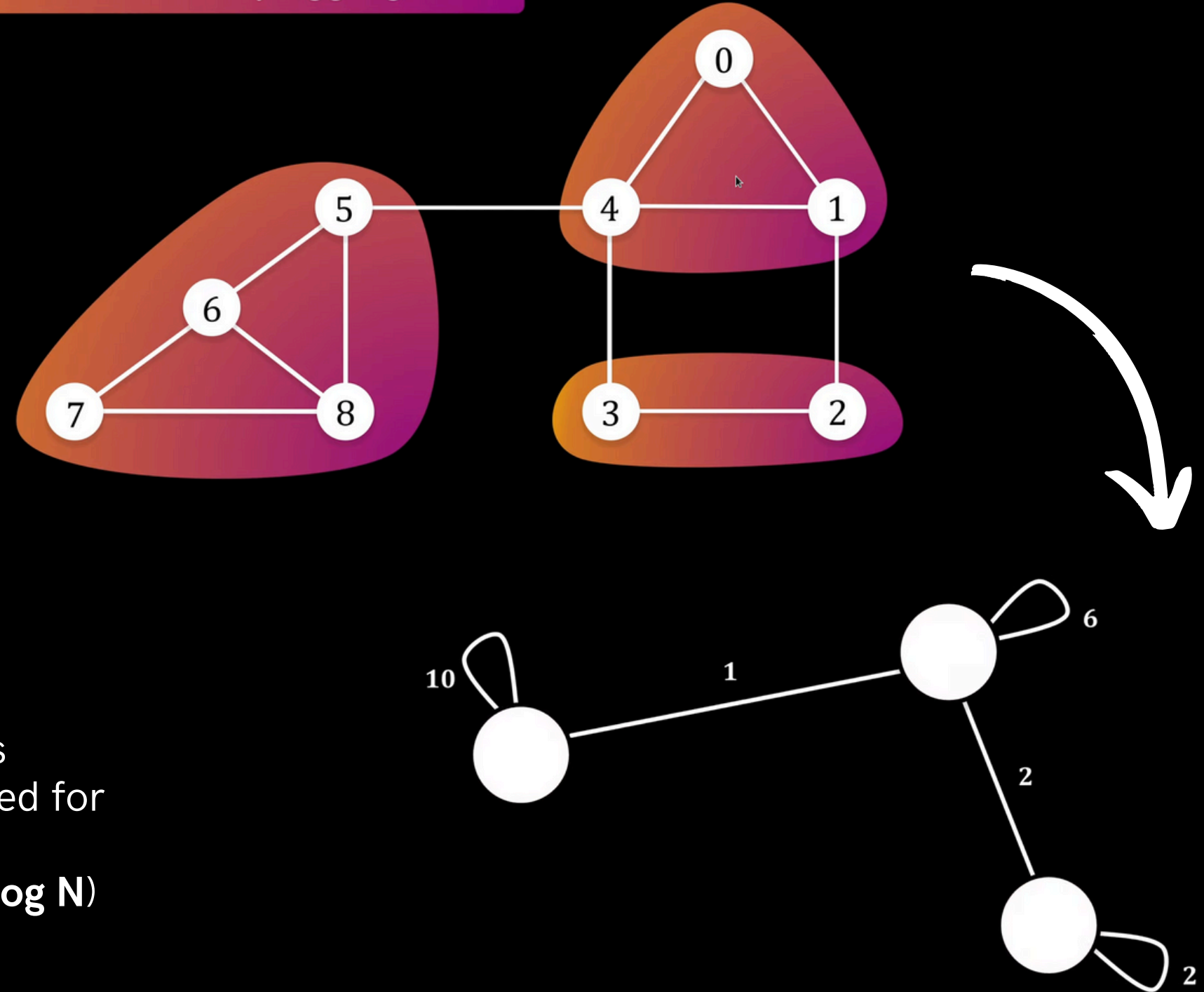
Phase 2: Graph Aggregation (Network Coarse Graining):

- Once Phase 1 stabilizes, all nodes belonging to the same community are collapsed (or merged) into a single, new entity called a "super-node".
- A new, smaller network is created based on these super-nodes, and the entire two-phase process restarts on this contracted graph.

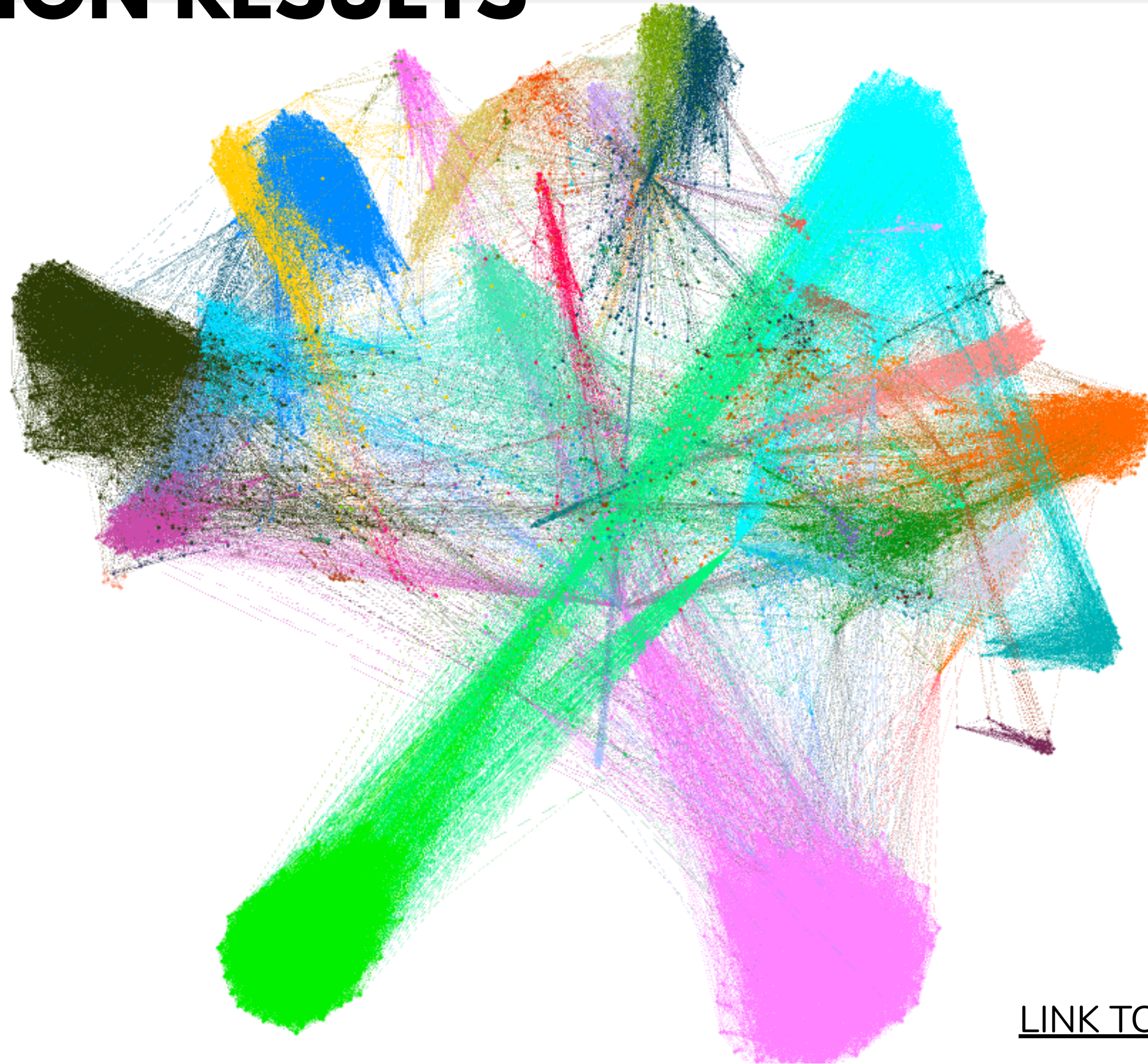
Time Complexity

- The most significant advantage of the Louvain algorithm is its remarkable speed and scalability, which is why it is widely used for large networks.
- The time complexity is typically cited as near-linear or $O(M \cdot \log N)$

Phase B: Community aggregation



SIMULATION RESULTS



[LINK TO OUR SIMULATION VIDEO](#)

**simulated on Gephi 0.1 on Linux using 17k+ connections of 4k+ facebook users using Fast Louvain Algorithm*

Inefficiency - Louvain's Computational Bottlenecks

The core Louvain algorithm is near-linear, but for networks with millions of edges, the constant factor and hidden costs become significant bottlenecks. The algorithm's iterative, greedy nature leads to **three** major sources of wasted computation:

- **Redundant Iteration Passes (Local Stability):** The algorithm re-traverses all nodes in every iteration of Phase 1. However, once a pair of communities reach a "local stable structure" (i.e., no modularity gain is possible between them), the calculation is repeated needlessly. This means repeated, unnecessary calculations of the Modularity Gain (ΔQ) for stable pairs.
- **Slow Convergence on Sparse Structures:** Many real-world networks contain sparse leaf nodes and local tree structures (long chains). Including these structurally unambiguous nodes in the main iteration wastes time and slows the convergence of the entire graph.
- **Wasted Final Moves (Early Stopping):** The stopping criterion requires zero node moves to halt the iteration. The final moves yield negligible modularity change but require a full pass over the entire graph.

Governing Equation: Modularity Gain ()

The decision to move a node is based on maximizing the gain, which requires the repeated calculation of this complex difference:

$$\Delta Q = \left[\frac{\sum_{in} + k_{i,in}}{2m} - \left(\frac{\sum_{tot} + k_i}{2m} \right)^2 \right] - \left[\frac{\sum_{in}}{2m} - \left(\frac{\sum_{tot}}{2m} \right)^2 \right]$$

The Failure of Modularity

Structural Limitation (Focus: Community Quality)

Even with perfect efficiency, the Louvain algorithm is fundamentally limited by the metric it optimizes: Modularity (Q)

1. The Resolution Limit: Modularity is calculated against a global null model (a random graph). This global comparison prevents the algorithm from detecting small, tightly-knit communities when they are weakly connected to the rest of the network.
 - Result: Small, genuine sub-communities are merged into larger clusters, hiding the network's finer structure.
2. Disconnected and Poorly Connected Communities: Louvain's Phase 1 is a purely greedy optimization. A move that increases Q (globally beneficial) might locally break a community's internal connectivity.
 - A node acting as a "bridge" in its own community may move, leaving the old community internally disconnected.

The Improvement : The Fast Louvain Mechanism

Our Strategy:

Improvement 1: Dynamic Iteration Optimization:

Goal: Eliminate ΔQ recalculations for stable communities.

Mechanism: The algorithm tracks which community pairs were unchanged in an iteration. Future passes skip considering node movements between these identified "local stable structures."

Improvement 2: Early Stopping Criterion:

- Goal: Avoid spending time on the marginal gains ΔQ in the final passes.
- Mechanism: Stop the Phase 1 iteration if the number of nodes that move is less than a preset stopping parameter (e.g 0.1% of total nodes).

Improvement 3: Splitting Local Tree Structures

- Goal: Remove redundant nodes from the costly iterative process to improve efficiency and partition effect.
- Mechanism: The algorithm identifies all tree-like components (leaf nodes, chains), removes them, runs the Louvain process on the smaller, dense core, and then reattaches the trees to the resulting communities.

Mathematical Justification for Tree Splitting

Removing these sparse structures is mathematically safe because the modularity change ΔQ caused by splitting or merging them is negligible . ($\Delta Q \approx 0$)

IMPROVEMENTS

01 Splitting Local Tree Structures

Standard Louvain:

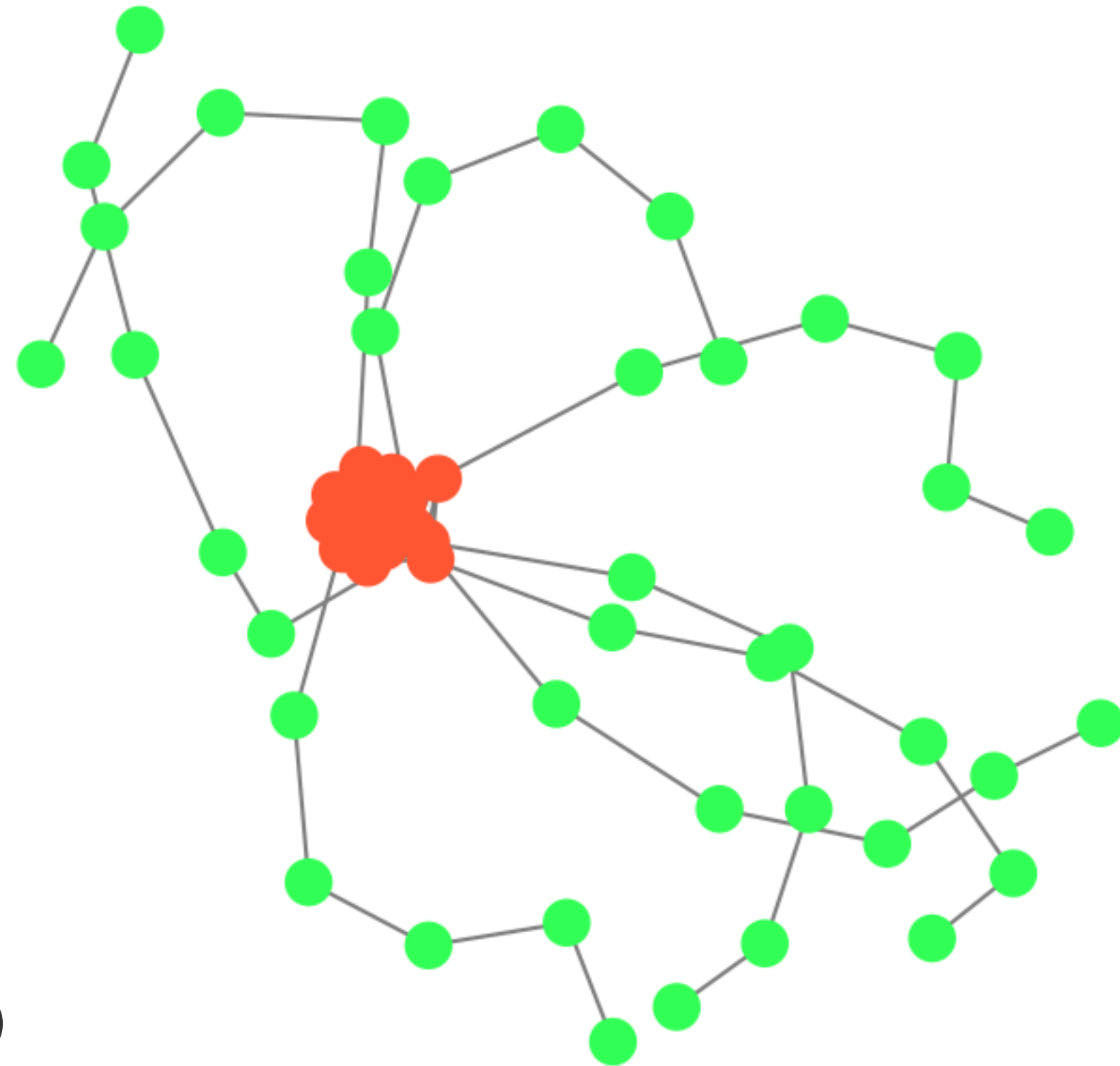
The algorithm **wastes** iterations calculating modularity for every single node in those **long chains** repeatedly, even though they will always just belong to the community of the node they are attached to.

Improvement:

Identify all tree-like components (chains) as preprocessing, remove them, run the Louvain process on the smaller, dense core, and then reattaches the trees to the resulting communities.

How?

- queue-based cascading removal strategy - $O(V+E)$
- reconstruct the complete partition using stack - $O(m)$



IMPROVEMENTS

02 Dynamic Iteration Optimization & 03 Early Stopping Criteria

Standard Louvain:

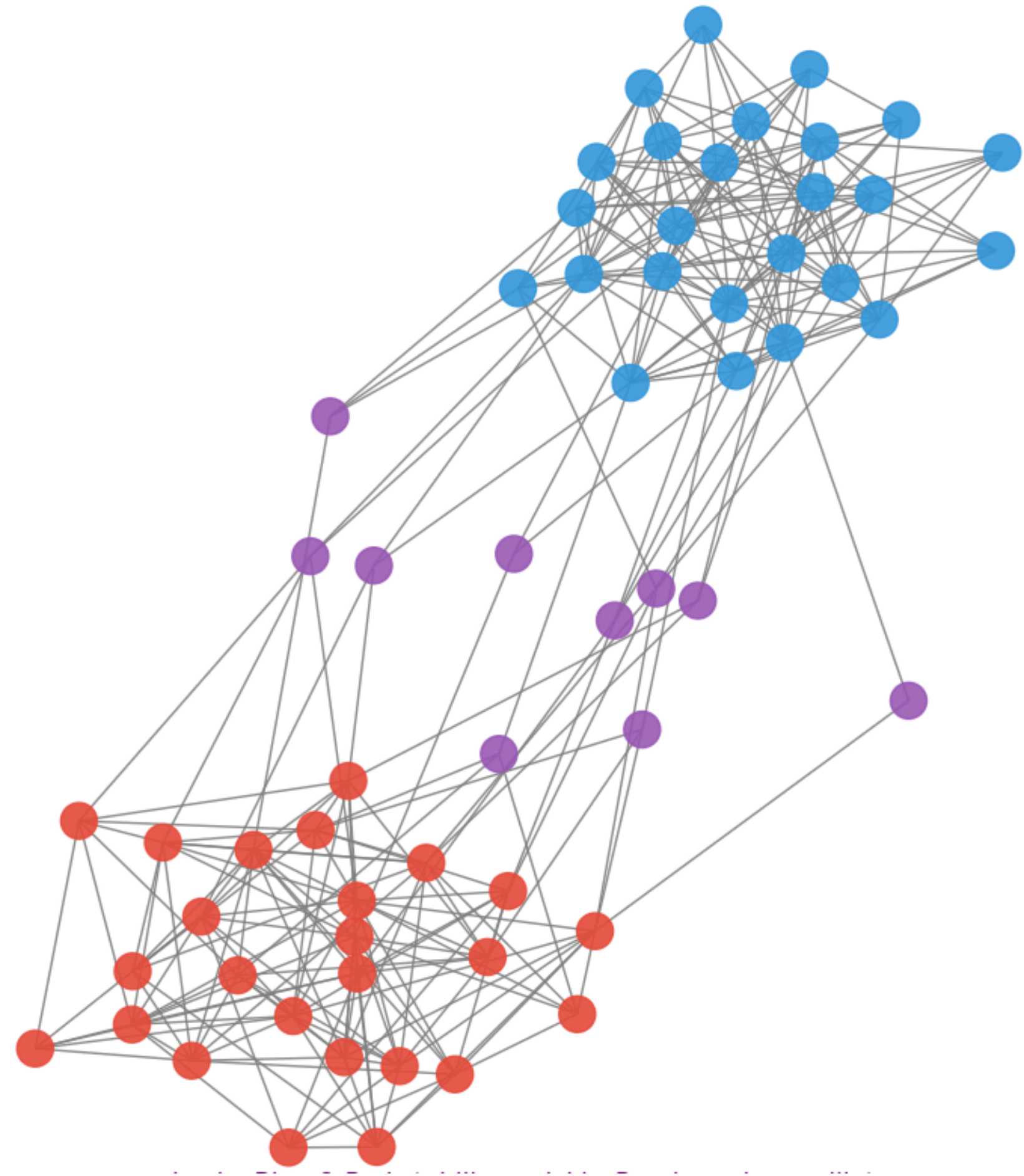
- every node checks every neighbor in every single pass, even if **nothing has changed** in that neighborhood since the last pass.
- Toward the end of the algorithm, the "core" of the communities is usually solid. However, there exist **"fuzzy" nodes** on the boundaries between communities

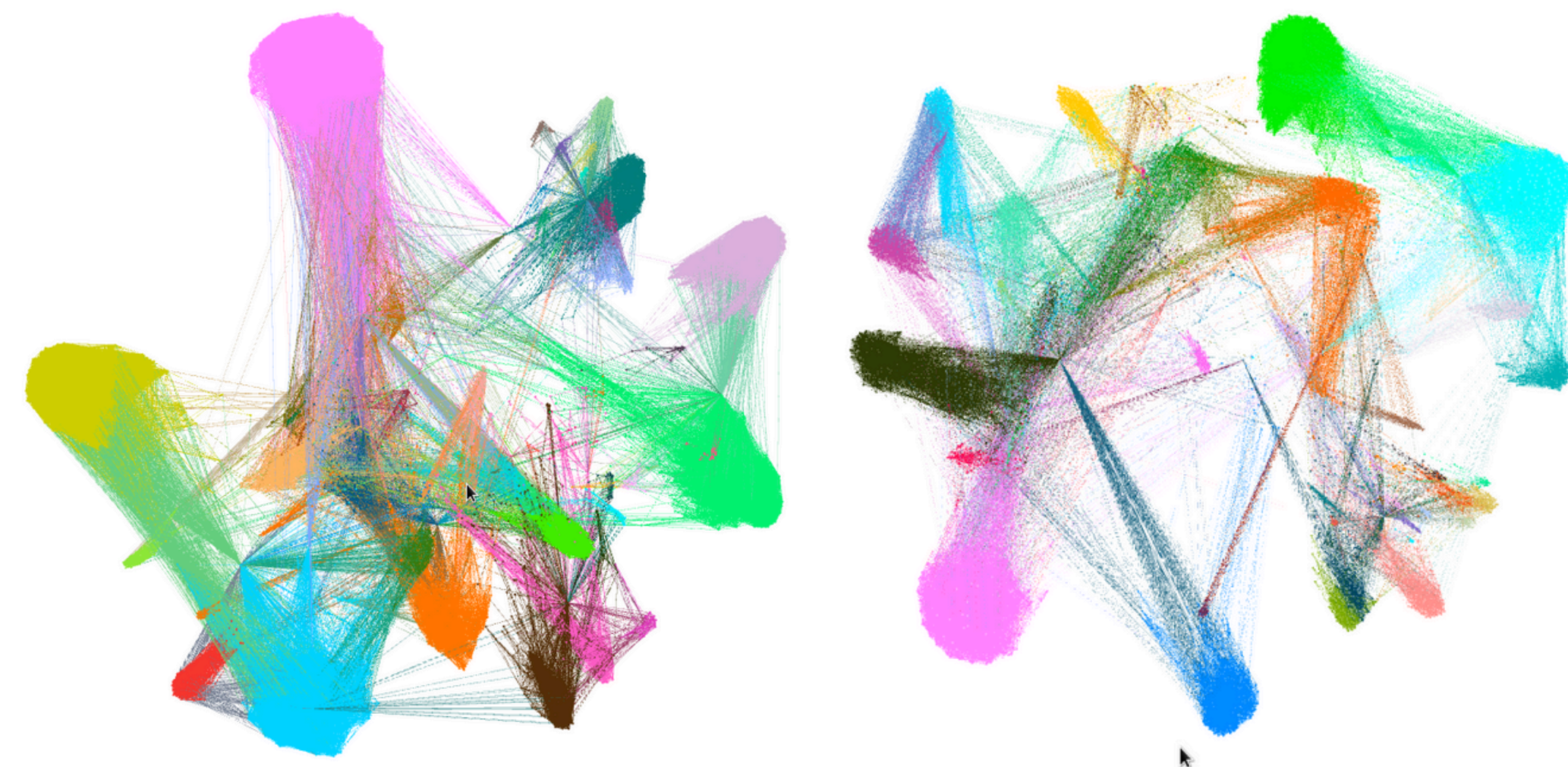
Improvements:

- skipping **redundant del Q** calculations for nodes within topologically stationary communities
- halting execution when the node migration rate drops below a **negligible threshold** rather than waiting for absolute stability

Complexity Impact:

- Standard: $O(k \cdot N)$ per pass
- Improved: $O(k \cdot N_{\text{active}})$ per pass





Standard vs Fast Louvain

Significant reductions in running time and number of steps with similar modularity

**simulated on Gephi 0.1 on Linux. 17k+ connections of 4k+ facebook users using Standard vs. Fast Louvain
 community colors are randomised hence cannot be compared in both the graphs

```

Loading 'facebook.txt'...
Graph loaded. Nodes: 4039. 2m = 176468
Graph loaded. 2m = 176468
=====

Starting [STANDARD] Run...
Running Phase 2: Community aggregation...
Phase 2: Aggregating 4039 nodes into 103 communities.
Running Phase 2: Community aggregation...
Phase 2: Aggregating 103 nodes into 72 communities.

Starting [FAST MODE] Run...
Running Phase 2: Community aggregation...
Phase 2: Aggregating 4039 nodes into 99 communities.
Running Phase 2: Community aggregation...
Phase 2: Aggregating 99 nodes into 63 communities.

=====
BENCHMARK RESULTS
=====

1. Standard Louvain:
  - Total Steps (Calc dQ): 232687
  - Modularity (Q): 0.86189
  - Wall Time (ms): 1146

2. Fast Louvain (Tree Split + Dynamic Iteration):
  - Total Steps (Calc dQ): 150255
  - Modularity (Q): 0.85817
  - Wall Time (ms): 739

-----
IMPROVEMENT:
  Saved 82432 calculation steps.
  Efficiency Gain: 35.43% fewer calculations.
=====
Results successfully written to communities.csv
  
```

Performance Impacts

The Fast Louvain algorithm achieves crazy performance improvements by reducing the number of operations required for convergence.

Dataset	Original Time (ms)	Fast Time (ms)	Old Modularity	New Modularity	Decrease in Number of Steps	Efficiency Gain (%)
Twitter	109624	94830	0.66774	0.68651	766975	13.78%
Facebook (small)	1196	765	0.86540	0.86889	44821	22.34%

Impact on Quality

The algorithm successfully maintains modularity (sometimes even increseases) while aggressively targeting speed.

- **Community Structure:** The sparse parts of the network become more correctly aggregated, leading to fewer small, meaningless micro-communities. The partition result is made more uniform, avoiding excessive aggregation in sparse areas.

Conclusions: Fast Louvain Performance

Massive Efficiency Gains: The Fast Louvain Algorithm (FLA) delivers a dramatic improvement in computational efficiency over the traditional Louvain Algorithm (LVA), achieving runtime reductions of 15%-30% on large-scale, sparse networks. This gain is primarily driven by optimizing the iterative process and pre-processing local tree structures.

Quality Maintained or Enhanced: Despite aggressively targeting speed, the FLA maintains or slightly improves the final Modularity (Q) score. Critically, its tree splitting strategy leads to a more uniform and refined partition, preventing excessive aggregation in sparse network areas and resulting in higher community aggregation overall.

Superior Scalability for Big Data: The FLA is a robust and practical solution for analyzing modern complex networks, which often contain millions of nodes and edges. Its ability to handle large-scale datasets while reducing computation time by nearly 20% establishes it as a superior and more scalable alternative to the classical LVA.

THANK YOU